

PROMPT FINANCE HACKATHON COST DATA PACK

23–24 May 2026 · Startup Sauna, Espoo · Two parquets + this README

AIVEN X AALTO PROMPT FINANCE HACKATHON — COST DATA PACK

May 23–24, 2026

This pack contains two sanitised parquet files derived from Aiven's production cloud-infrastructure cost-to-serve data, plus this README. It is built for the Prompt Finance Hackathon and should not be redistributed outside the event.

FILES

FILE	ROWS	SIZE	PURPOSE
<code>aiven_usage.parquet</code>	~4.1M	~57 MB	Daily, line-item cost-to-serve data (sanitised)
<code>cross_cloud_list_prices.parquet</code>	~130	~8 KB	Cross-cloud list-price ratios per SKU family
<code>README.md</code>	—	—	This file

THE FRAMING

Aiven runs a managed database / streaming platform across multiple public cloud providers. Cost-to-serve — the infrastructure bill Aiven pays the underlying clouds for running customer workloads — is one of the single largest lines in our gross margin. Small improvements in cloud-pricing decisions, commitments, and routing compound into material P&L impact.

This dataset is designed so a finance/strategy team (you) can play with the shape of that spend without seeing the raw confidential numbers.

WHAT THIS DATA IS (AND IS NOT)

Is: - 18 months of daily, line-item-level infrastructure cost-to-serve records. - A sanitised view of real production billing data — shapes and relationships preserved, absolutes obscured.

Is NOT: - Customer-identifiable. No account / customer / tenant / project / service IDs. - Actual cloud-provider list prices in dollars. The cross-cloud catalog expresses prices as **ratios** within each (geo, SKU family, unit) group — no absolute USD. - A reliable reconstruction of Aiven's actual cost-to-serve. The `amount` column has been scaled and noised; sums will NOT equal Aiven's real spend. - Aiven's internal R&D infrastructure spend. Only customer-serving rows are included.

DATA DICTIONARY

`aiven_usage.parquet`

COLUMN	TYPE	DESCRIPTION
<code>line_id</code>	int64	Synthetic sequential row id
<code>status_date</code>	datetime	Day the cost was incurred (UTC)
<code>month</code>	datetime	First day of the invoice month for that cost
<code>cloud</code>	string	Anonymised cloud provider: one of <code>cloud_a</code> .. <code>cloud_d</code> (4 distinct providers; mapping not disclosed)
<code>geo_bucket</code>	string	Coarse geographic region: <code>eu-north</code> , <code>eu-west</code> , <code>us-east</code> , <code>us-west</code> , <code>apac</code> , <code>other</code>
<code>service_category</code>	string	High-level Aiven-service grouping: <code>database-relational</code> , <code>database-nosql</code> , <code>streaming</code> , <code>search</code> , <code>analytics</code> , <code>observability</code>
<code>sku_family</code>	string	Cloud SKU family bucket: see "SKU family taxonomy" below
<code>cost_category</code>	string	Coarse cost-driver category (from source: <code>Compute</code> , <code>Data Transfer Out</code> , <code>Persistent Disk Storage</code> , ...)
<code>cost_sub_category</code>	string	Finer cost-driver sub-category
<code>cost_category_group</code>	string	Top-level grouping: <code>Compute</code> , <code>Storage</code> , <code>Network</code> , <code>Other</code>
<code>amount</code>	float64	Sanitised consumed quantity in the given <code>unit</code> . Scaled and noised — absolute totals are not Aiven's actuals
<code>unit</code>	string	Unit of <code>amount</code> : <code>hour</code> , <code>gib</code> , <code>gib-hour</code> , <code>gib-month</code> , <code>gb</code> , <code>gb-month</code> , <code>count</code> , <code>month</code> , ...

cross_cloud_list_prices.parquet

COLUMN	TYPE	DESCRIPTION
cloud	string	Anonymised cloud provider (same encoding as in the usage file)
geo_bucket	string	Geographic region bucket (same encoding)
sku_family	string	SKU family (same encoding)
unit	string	Unit the price is per (same encoding)
p25_ratio	float	25th percentile of per-SKU list-price ratios in this group
p50_ratio	float	Median list-price ratio (the "headline" number)
p75_ratio	float	75th percentile

For each (geo_bucket, sku_family, unit) group, one cloud is set as the baseline (median ratio $\approx$ 1.0) and other clouds are expressed as ratios relative to that baseline. A small per-row noise factor has been applied to obscure exact relationships.

A ratio of 1.40 means: in this region + SKU family + unit, this cloud's list price is on the order of 40% more expensive than the baseline cloud, per unit consumed.

HOW TO JOIN THE TWO FILES

There is no row-level join: the usage file is line-item, the catalog is one row per (cloud, geo_bucket, sku_family, unit) group. They join on (cloud, geo_bucket, sku_family, unit):

```
import pandas as pd
u = pd.read_parquet("aiven_usage.parquet")
p = pd.read_parquet("cross_cloud_list_prices.parquet")
joined = u.merge(p, on=["cloud", "geo_bucket", "sku_family", "unit"], how="left")
# joined.amount * joined.p50_ratio gives you a list-price-weighted "synthetic spend"
```

Because list prices are stripped of absolute USD, you cannot reconstruct dollar spend. You CAN reason about **relative** cloud-cost trade-offs and **scaled quantity** dynamics — which is most of what matters for the angles below.

HOW THIS DATA HAS BEEN SANITISED

Aiven has sanitised this dataset before sharing. The headline facts:

- All customer, tenant, employee, project, and service identifiers are removed.
- Cloud providers are anonymised; the mapping is not disclosed.
- Regions are grouped into coarse geographic buckets.

- SKU descriptions are grouped into ~18 family buckets; vendor proper nouns are scrubbed from any free-text columns.
- Consumption volumes (`amount`) have been scaled and noised. Absolute totals do not match Aiven's real spend.
- The cross-cloud price catalog is ratio-only — no absolute USD anywhere.
- Production-only scope. Internal R&D infrastructure spend is excluded.

Please do not attempt to reverse the sanitisation, infer the cloud mapping, or publish findings that claim to represent Aiven's actual figures. Shapes and relationships are real; absolutes are not.

FOUR EXAMPLE ANGLES

The dataset can support all four of the angles below — or, much more interesting, an angle we haven't thought of. Pick one, combine, or invent your own.

1. Forecasting cost-to-serve

Build a daily / weekly forecast for total `amount` by `cloud × service_category × geo_bucket` over the next 90 days. Hold out the last 30 days as a test window. Score with MAPE on monthly totals. Bonus: separate seasonal vs. trend vs. drift in the data.

The data has real (if synthetic) drift to learn — naïve last-month-equals-next-month forecasts will lose to even simple exponential smoothing.

2. Commitment optimisation

Given the volume of `compute-general`, `compute-cpu-optimized`, and `compute-memory` consumption per `(cloud × geo_bucket)`, decide how much 1-year / 3-year commitment to take. Assume a 1-year commitment locks in ~30% discount on covered usage; 3-year ~55%. Penalty for over-committing: idle paid capacity.

Build an optimiser (or LP) that takes the per-month consumption forecast and outputs a commitment-tier schedule. The `commitment-discount` SKU family in the data shows what we currently do — your job is to do it better.

3. Price-performance, productised

Use `cross_cloud_list_prices.parquet`. For each `(geo_bucket, sku_family, unit)`, identify the cheapest cloud and the spread. Then go to `aiven_usage.parquet`: where are we running large workloads on the expensive cloud when a cheaper cloud in the same `geo_bucket` has the same SKU family? Quantify the "if you moved the workload" annualised savings (in ratio terms, not USD).

Turn this into a packaged recommendation: a one-page report a Solutions Architect can use to talk to a customer about cloud choice. Use the prompt-engineering angle: get an LLM to write the recommendation given the structured numbers.

4. Deal optimisation: matching commitments to renewals

This is the hardest angle. The usage file has month-level grain. Imagine each cloud × geo_bucket bundle is a "deal" with a commitment renewal date.

Build a model that: (a) clusters service workloads by usage-shape (peaky, steady, growing, declining), (b) for each cluster, picks the best commitment depth (no commitment / 1-year / 3-year / mix), and (c) shows the cumulative ratio-weighted cost saving over the next 18 months versus a flat "no commitments anywhere" baseline.

CAVEATS AND HONESTY

- **4 anonymised clouds in this dataset.** Aiven runs on more clouds in production, but only 4 met the data-shape requirements (per-unit consumption available) for inclusion here.
- **amount = 0 rows exist.** Many transfer-related SKUs record zero bytes on a given day — those are legitimate billing lines and we kept them.
- **The catalog has 130 rows.** Cross-cloud comparability is sparse: many (geo_bucket, sku_family, unit) groups only have one cloud with enough SKUs to compute a median. Single-cloud groups are filtered out. A missing row means at least one cloud in that group doesn't sell that SKU family in that geo.
- **Absolute volumes are obscured; shapes are preserved.** Year-over-year growth, monthly seasonality, and cloud-mix proportions should be roughly correct directionally; absolute monthly totals are not.

CONTACT

For data issues during the hackathon: ping Stijn on Discord.